CO1 ASSESSMENT TOOL 1

NAME: YAMUNA K

REG NO: 192511013

COURSE CODE: CSA1735

COURSE NAME: ARTIFICIAL INTELLIGENCE

ARTIFICIAL INTELLIGENCE – ANALYTICAL PROBLEM SOLVING

1. Route Finding with A* Search

Aim:

To find the shortest path between a starting location and a destination using the A* search algorithm.

Problem Description:

A* search is an informed search algorithm used to find the best path from a starting point to a goal. It uses the actual cost of the path and the estimated cost to reach the goal.

Formula:

$f(n) = g(n) + h(n)$

Where:

- g(n) – Actual cost from the starting point to the current node.
- h(n) – Estimated cost from the current node to the goal.
- f(n) – Total estimated cost of the path.

Algorithm:

1. Start from the initial location.
2. Calculate the f(n) value for each available node.
3. Select the node with the lowest f(n) value.
4. Expand the selected node and calculate the costs of its neighboring nodes.
5. Repeat the process until the destination is reached.
6. Display the shortest path and its total cost.

Example:

Consider the following routes:

- A to B = 2
- A to C = 4
- B to D = 3
- C to D = 1
- D to E = 2

Starting point: A

Destination: E

Solution:

The possible paths are:

Path 1: A → B → D → E

Total cost = 2 + 3 + 2 = 7

Path 2: A → C → D → E

Total cost = 4 + 1 + 2 = 7

Result:

The A* search algorithm finds an optimal path from the starting location to the destination. In this example, both paths have the same minimum cost of 7.

2. Food Delivery Route Optimisation Using Local Search

Aim:

To optimise the delivery route of a food delivery agent using a local search technique.

Problem Description:

A food delivery agent must deliver food to multiple customers. The objective is to find a route that minimises the total distance travelled and reduces delivery time.

Local search improves a current solution by making small changes to it and selecting a better solution.

Algorithm:

1. Identify the restaurant and all customer locations.
2. Create an initial delivery route.
3. Calculate the total distance of the route.
4. Generate a neighbouring route by changing the order of customer visits.
5. Calculate the distance of the new route.
6. If the new route is shorter, replace the current route.
7. Repeat until no better route is found or the stopping condition is reached.
8. Display the optimised route and total distance.

Example:

A food delivery agent must deliver food from restaurant R to three customers A, B and C.

Initial Route:

R → A → B → C → R

Total distance = 18 km

After applying local search:

R → B → A → C → R

Total distance = 14 km

Distance saved = 18 − 14 = 4 km

Advantages:

1. Reduces the total delivery distance.
2. Saves fuel and travel time.
3. Helps improve delivery efficiency.
4. Can handle different possible delivery routes.

Result:

Local search optimises the food delivery route by finding a shorter route of 14 km instead of the initial 18 km, based on the example.

3. Uninformed Search Comparison (BFS, DFS and IDS)

Aim:

To compare Breadth-First Search (BFS), Depth-First Search (DFS) and Iterative Deepening Search (IDS) based on their working principles, memory usage and completeness.

Problem Description:

Uninformed search algorithms explore a search space without using heuristic information about how close a node is to the goal.

The three commonly used uninformed search algorithms are BFS, DFS and IDS.

1. Breadth-First Search (BFS)

BFS explores nodes level by level. It uses a queue to store the nodes waiting to be explored.

Algorithm:

1. Insert the starting node into the queue.
2. Remove the first node from the queue.
3. Check whether it is the goal node.
4. If not, add its unvisited neighbouring nodes to the queue.
5. Repeat until the goal is found or the queue becomes empty.

2. Depth-First Search (DFS)

DFS explores one branch as deeply as possible before backtracking. It uses a stack or recursion.

Algorithm:

1. Start from the initial node.
2. Visit an unvisited neighbouring node.
3. Continue exploring deeper nodes.
4. If no unvisited neighbour exists, backtrack.
5. Repeat until the goal is found or all reachable nodes are explored.

3. Iterative Deepening Search (IDS)

IDS repeatedly performs depth-limited search, increasing the depth limit each time.

Algorithm:

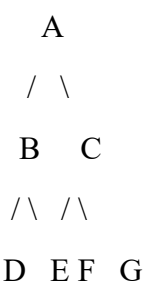
1. Set the depth limit to zero.
2. Perform depth-limited search from the starting node.
3. Check whether the goal is found.
4. If not, increase the depth limit by one.
5. Repeat until the goal is found or the search limit is reached.

Comparison Table:

Feature	**BFS**	**DFS**	**IDS**
Search method	Level by level	Deep into one branch	Repeated depth-limited search
Data structure	Queue	Stack	Stack or recursion
Completeness	Yes, for finite branching	Not always	Yes, for finite branching
Optimality	Yes, for equal step costs	No	Yes, for equal step costs
Memory usage	High	Low	Low compared with BFS
Main advantage	Finds the shallowest solution	Uses less memory	Combines BFS completeness with DFS-like memory efficiency

Example:

Consider the search tree:



Starting node: A

Goal node: F

- BFS visits: A → B → C → D → E → F
- DFS, exploring left to right, visits: A → B → D → E → C → F
- IDS searches with depth limits 0, 1 and 2 until it finds F.

Result:

BFS is suitable for finding the shallowest solution, DFS is useful when memory is limited, and IDS is suitable when the solution depth is unknown and memory efficiency is important.

4. Smart Traffic Signal Controller (Agent Design)

Aim:

To design an intelligent agent that controls traffic signals based on real-time traffic conditions.

Problem Description:

Traditional traffic signals often operate using fixed timings. A smart traffic signal controller uses information from sensors or cameras to adjust signal timings according to the traffic density on each road.

Agent Design:

1. Performance Measure

- Reduce traffic congestion.
- Minimise vehicle waiting time.
- Improve traffic flow.
- Give priority to emergency vehicles when appropriate.
- Maintain pedestrian safety.

2. Environment

The environment includes roads, junctions, vehicles, pedestrians, traffic signals and emergency vehicles.

3. Sensors

The agent receives information from:

- Traffic cameras.
- Vehicle detection sensors.
- Pedestrian crossing buttons or detectors.
- Emergency vehicle detection systems.

4. Actuators

The agent controls:

- Red traffic lights.
- Yellow traffic lights.
- Green traffic lights.
- Pedestrian crossing signals.

5. Agent Program

1. Collect traffic data from sensors.
2. Calculate the traffic density on each road.
3. Identify emergency vehicles and pedestrian crossing requests.
4. Select the appropriate signal phase based on traffic conditions and safety rules.
5. Adjust the green signal duration within predefined limits.
6. Change signals safely using the required transition intervals.
7. Repeat the process continuously.

Example:

At a four-road junction:

- North road: High traffic density.
- South road: Medium traffic density.
- East road: Low traffic density.
- West road: Low traffic density.

The controller may allocate more green time to the north road while maintaining safe signal transitions and considering pedestrian requests.

Advantages:

1. Reduces unnecessary waiting time.
2. Improves traffic flow.
3. Reduces congestion.
4. Supports emergency vehicle priority.
5. Improves overall road safety when designed with appropriate safeguards.

Result:

The smart traffic signal controller uses sensor information to adapt signal timings to traffic conditions while following safety constraints.

5. OLA Cab Booking (Problem-Solving Agent and Pseudocode)

Aim:

To design a problem-solving agent for an OLA cab booking system that finds a suitable cab and assigns it to a passenger.

Problem Description:

A cab booking system receives a passenger's pickup location and destination. The system searches for available cabs, evaluates their distance and estimated arrival time, and assigns a suitable cab to the passenger.

Problem-Solving Agent:

A problem-solving agent identifies the goal, formulates the problem, searches for a solution and executes the selected actions.

Components:

1. Initial State: Passenger requests a cab.
2. Goal State: A suitable available cab is assigned to the passenger.
3. Actions: Search for cabs, calculate distances, estimate arrival times and assign a cab.
4. Path Cost: Distance, estimated pickup time or another defined service cost.
5. Solution: Select a suitable cab and confirm the booking.

Algorithm:

1. Receive the pickup and destination locations.
2. Search for available cabs near the pickup location.
3. If no cab is available, inform the passenger.
4. Calculate the estimated arrival time for each available cab.

5. Select a suitable cab based on estimated arrival time and other booking rules.
6. Assign the selected cab to the passenger.
7. Display the cab details and estimated arrival time.
8. Confirm the booking.

Pseudocode:

```
BEGIN

INPUT pickup_location
INPUT destination

available_cabs = FIND_AVAILABLE_CABS (pickup_location)

IF available_cabs is EMPTY THEN
   DISPLAY "No cabs available"
ELSE
   FOR each cab IN available_cabs DO
      cab.ETA = CALCULATE_ETA (cab, pickup_location)
   END FOR

   selected_cab = cab with the lowest ETA
                among suitable available cabs

   ASSIGN selected_cab TO passenger

   DISPLAY "Cab Booking Successful"
   DISPLAY selected_cab.details
   DISPLAY selected_cab.ETA
END IF

END
```

Example:

Passenger pickup location: Saveetha University

Destination: Chennai Airport

Available cabs:

- Cab A: Estimated arrival time of 8 minutes.
- Cab B: Estimated arrival time of 5 minutes.
- Cab C: Estimated arrival time of 10 minutes.

Solution:

Cab B is selected because it has the lowest estimated arrival time of 5 minutes, assuming all three cabs are otherwise suitable and available.

Advantages:

1. Reduces the time required to find a cab.
2. Helps assign nearby available drivers.
3. Provides an estimated arrival time.
4. Improves booking efficiency.
5. Reduces manual effort.

Result:

The problem-solving agent identifies available cabs, compares their estimated arrival times and assigns a suitable cab to the passenger.